Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №47

ТЕМА: Менеджери геометрії (pack, grid, place) та адаптивна верстка у контейнерах CTkFrame і CTkScrollableFrame.

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- ознайомитися з трьома менеджерами геометрії CustomTkinter: pack, grid, place;
- вивчити принципи роботи кожного менеджера та його ключові параметри;
- ознайомитися з контейнерами CTkFrame та CTkScrollableFrame;
- навчитися будувати адаптивну верстку інтерфейсу за допомогою вкладених контейнерів.

Теоретичні відомості

1. Навіщо потрібні менеджери геометрії

Створення віджета (наприклад, CTkButton(app, text='Кнопка')) саме по собі не робить його видимим на екрані — потрібно ще й указати, де саме в батьківському контейнері він має розташовуватися. Для цього в CustomTkinter (як і в звичайному tkinter) використовують один із трьох менеджерів геометрії: pack, grid або place. Важливе правило: в межах одного контейнера не можна одночасно використовувати різні менеджери геометрії для різних його дочірніх віджетів — це може призвести до непередбачуваної поведінки інтерфейсу чи навіть до «зависання» вікна.

2. Менеджер pack()

pack() — найпростіший менеджер: він розміщує віджети послідовно один за одним, «упаковуючи» їх у контейнер із заданого боку (за замовчуванням — згори вниз). Основні параметри:

Параметр	Призначення
side	бік розміщення: 'top' (за замовчуванням), 'bottom', 'left', 'right'
pady / padx	зовнішній відступ по вертикалі / горизонталі (у пікселях)
fill	розтягування віджета: 'x', 'y' або 'both'
expand	чи повинен віджет займати весь доступний вільний простір

```
import customtkinter as ctk
ctk.set_appearance_mode('Dark')
app = ctk.CTk()
app.geometry('300x250')

ctk.CTkLabel(app, text='Верх').pack(side='top', pady=10)
ctk.CTkLabel(app, text='Ліво').pack(side='left', padx=10)
ctk.CTkLabel(app, text='Право').pack(side='right', padx=10)
ctk.CTkLabel(app, text='Низ').pack(side='bottom', pady=10)

app.mainloop()
```

pack() зручний для простих, лінійних інтерфейсів (список елементів один під одним), але стає незручним, коли потрібна складніша, табличного вигляду верстка — для цього краще підходить grid().

3. Менеджер grid()

grid() розміщує віджети у вигляді таблиці — за номером рядка (row) і стовпця (column), нумерація яких починається з 0. Цей менеджер значно зручніший для побудови форм (як-от анкет реєстрації), де елементи природно вирівнюються в колонки:

```
frame = ctk.CTkFrame(app)
frame.pack(padx=20, pady=20)

ctk.CTkLabel(frame, text="Ім'я:").grid(row=0, column=0, padx=10,
pady=10, sticky='e')
ctk.CTkEntry(frame).grid(row=0, column=1, padx=10, pady=10)
ctk.CTkLabel(frame, text='Вік:').grid(row=1, column=0, padx=10,
pady=10, sticky='e')
ctk.CTkEntry(frame).grid(row=1, column=1, padx=10, pady=10)
```

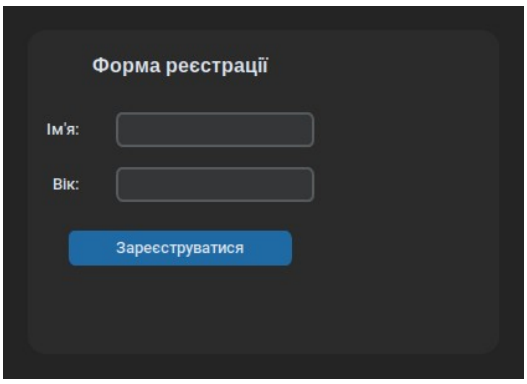


Рис. 1. Форма реєстрації, побудована за допомогою менеджера grid() у контейнері CTkFrame

Параметр sticky визначає, до якого краю комірки таблиці «прилипає» вміст, якщо комірка більша за сам віджет: 'e' (east, схід) — праворуч, 'w' (west) — ліворуч, 'n', 's' — угору, вниз відповідно; ці літери можна комбінувати (наприклад, 'ew' розтягує віджет по всій ширині комірки).

4. Менеджер place()

place() дозволяє вказати точні координати віджета (у пікселях або у відсотках від розміру контейнера). Цей спосіб дає максимальний контроль над розташуванням, але водночас найменш гнучкий: при зміні розміру вікна віджети, розміщені через place(), не підлаштовуються автоматично, як це частково відбувається з pack() чи grid().

```
label = ctk.CTkLabel(app, text='Точна позиція')
label.place(x=50, y=30)

button = ctk.CTkButton(app, text='Кнопка')
button.place(relx=0.5, rely=0.8, anchor='center')
```

Параметри relx і rely задають відносне положення (від 0 до 1) відносно розмірів контейнера, а anchor='center' означає, що вказана точка є центром віджета, а не його верхнім лівим кутом. place() варто використовувати помірно — переважно для елементів, яким справді потрібне точне позиціювання (наприклад, водяний знак чи індикатор у кутку), а не як основний спосіб побудови всього інтерфейсу.

5. Контейнер CTkFrame

CTkFrame — це «панель» або «контейнер», який групує кілька віджетів разом і сам може мати власний фон, заокруглення кутів і рамку. Використання фреймів дозволяє логічно розділити інтерфейс на змістовні блоки (наприклад, «блок введення даних», «блок кнопок дій», «блок результату»), кожен з яких можна стилізувати й розташовувати окремо від інших.

```
top_frame = ctk.CTkFrame(app, corner_radius=15)
top_frame.pack(padx=20, pady=(20, 10), fill='x')
ctk.CTkLabel(top_frame, text='Верхній блок').pack(pady=10)

bottom_frame = ctk.CTkFrame(app, corner_radius=15)
bottom_frame.pack(padx=20, pady=(10, 20), fill='both', expand=True)
ctk.CTkButton(bottom_frame, text='Дія').pack(pady=10)
```

Важливо розуміти, що менеджер геометрії, застосований до самого фрейму (наприклад, top_frame.pack(...)), і менеджер геометрії, застосований до віджетів усередині фрейму (наприклад, ctk.CTkLabel(top_frame, ...).pack(...)), — це дві незалежні системи координат: перша визначає, де фрейм розташований у батьківському вікні, друга — де віджети розташовані всередині самого фрейму.

6. Контейнер CTkScrollableFrame

Якщо вміст контейнера може бути більшим за видиму область вікна (наприклад, довгий список елементів), звичайний CTkFrame не дозволить проскролити вміст — частина елементів просто буде невидимою. Для таких

випадків призначений CTkScrollableFrame — контейнер з вбудованою вертикальною прокруткою, який використовується так само, як і звичайний CTkFrame, але автоматично додає повзунок прокрутки, коли вміст не вміщується по висоті:

```
scroll_frame = ctk.CTkScrollableFrame(app, width=300, height=200,
corner_radius=12)
scroll_frame.pack(padx=15, pady=15, fill='both', expand=True)

for i in range(15):
    ctk.CTkLabel(scroll_frame, text=f'Елемент {i + 1}').pack(pady=5)
```

У цьому прикладі 15 міток точно не вміщаються у видиму висоту 200 пікселів, тому CTkScrollableFrame автоматично показує смугу прокрутки, дозволяючи користувачу переглянути всі елементи.

Хід роботи

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Кількість міток у завданні 5 залежить від N (mod — остача від ділення):

Завдання	Дані варіанта	Приклад N = 7
5	n = 20 + N mod 5	n = 22

б) Кожне завдання — окремий файл Прізвище_ПР47_зN.py (наприклад, Ivanenko_PR47_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Для кожного вікна додайте до звіту скріншот із видимим заголовком вікна. Імпорт — import customtkinter as ctk; програма завершується app.mainloop(). Тексти елементів (кнопок, міток, повідомлень у консоль) — точно як в умові. Не змішуйте pack() і grid() в одному контейнері.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР47_з1.py. Розмістіть за допомогою pack() чотири мітки з різних боків вікна (top, bottom, left, right) із текстами «Верх», «Низ», «Ліво», «Право».

   Приклад: вікно 400×300: мітки по краях, фіксовані розміри не потрібні.

2. У CTkFrame за допомогою grid() побудуйте форму з двома парами «мітка + поле»: «Ім'я» і «Прізвище»; мітки вирівняйте праворуч (sticky='e').

   Приклад: мітки в стовпці 0, поля в стовпці 1; рядки 0 і 1.

3. За допомогою place() розмістіть підпис «Центр» по центру вікна (relx=0.5, rely=0.5, anchor='center') і кнопку «OK» у правому нижньому куті (relx=0.95, rely=0.95, anchor='se').

   Приклад: при зміні розміру вікна підпис лишається по центру.

4. Розбийте вікно на два CTkFrame (верхній і нижній) з різними corner_radius і розмістіть у кожному по два-три віджети.

   Приклад: верхній блок — радіус 10, нижній — радіус 25.

5. Створіть CTkScrollableFrame і заповніть його в циклі n мітками «Елемент k» (n за вашим варіантом); переконайтеся, що прокручування працює.

   Приклад (N = 7): n = 22: «Елемент 1» … «Елемент 22».

6. Завдання підвищеної складності*. Комбінований інтерфейс: зовнішній CTkFrame з grid()-формою (поле й кнопка «Додати») зверху та CTkScrollableFrame знизу зі списком із 5 заздалегідь заданих записів «Запис k».

   Приклад: у списку 5 записів, форма не зникає під час прокручування.

Критерії оцінювання

Складова	**Оцінювання**	**Макс. балів**
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, pack() і grid() в одному контейнері, не вказано sticky, фіксовані координати замість relx/rely, прокручування не працює) — 50%; програма не працює або не здана — 0.

Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити менеджер геометрії чи кількість елементів). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Чому не можна змішувати pack() і grid() для віджетів одного й того самого контейнера?
2. Якими параметрами задається положення віджета в grid()?
3. Що означає параметр sticky і які значення він може приймати?
4. У чому основна відмінність place() від pack() і grid() за гнучкістю та контролем розташування?
5. Для чого призначений CTkScrollableFrame і чим він відрізняється від звичайного CTkFrame?